BAIL security

Katana
SushiStaker

FINAL REPORT

February ‘2026

Disclaimer:

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

The content of this assessment is not an investment. The information provided in this report is for general informational purposes only and is not intended as investment, legal, financial, regulatory, or tax advice. The report is based on a limited review of the materials and documentation provided at the time of the audit, and the audit results may not be complete or identify all possible vulnerabilities or issues. The audit is provided on an "as-is," "where-is," and "as-available" basis, and the use of blockchain technology is subject to unknown risks and flaws.

The audit does not constitute an endorsement of any particular project or team, and we make no warranties, expressed or implied, regarding the accuracy, reliability, completeness, or availability of the report, its content, or any associated services or products. We disclaim all warranties, including the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We assume no responsibility for any product or service advertised or offered by a third party through the report, any open-source or third-party software, code, libraries, materials, or information linked to, called by, referenced by, or accessible through the report, its content, and the related services and products. We will not be liable for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract.

The contract owner is responsible for making their own decisions based on the audit report and should seek additional professional advice if needed. The audit firm or individual assumes no liability for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract. The contract owner agrees to indemnify and hold harmless the audit firm or individual from any and all claims, damages, expenses, or liabilities arising from the use or reliance on the audit report or the smart contract.

By engaging in a smart contract audit, the contract owner acknowledges and agrees to the terms of this disclaimer.

1. Project Details

Important:
Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Katana - SushiStaker - Audit Report
Website	app.katana.network
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/katana-network/sushi-v3-lp-staking-offchainAccounting/blob/1790d736438d773c4f6963ef6baf24040025fd08/src/SushiStaker.sol
Resolution 1	https://github.com/katana-network/sushi-v3-lp-staking-offchainAccounting/blob/591e63878c626a20d2e97e3fd8947667f29431d2/src/SushiStaker.sol

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no changes made)	Failed resolution	Open
High	1	1				
Medium						
Low	4	1		3		
Informational	4	1		3		
Governance	1			1		
Total	10	3		7		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

SushiStaker

The SushiStaker contract is a staking contract for NFPM (NonfungiblePositionManager) positions which are represented as an ERC721 tokenId, reflecting an underlying CLAMM position with its corresponding traits (tickLower, tickUpper, liquidity, ...).

Whenever a tokenID has been staked, any accrued fees (token0, token1) are forfeited and will be claimed towards the feeCollector address. The incentive behind tokenID staking is the distribution of KATANA tokens which will be handled by merkl.xyz using off-chain calculations, which is not part of the audit scope.

Appendix: Stake Mechanism

Users can stake their tokenIDs via the stake function which then transfers the tokenId into the SushiStaker contract and triggers the onERC721Received function which claims fees (token0/token1) on behalf of the owner and accounts for staking.

Appendix: Unstake Mechanism

Users can unstake their tokenIds via the unstake function which then claims all due fees towards the feeCollector address and transfers the tokenId back to the original owner.

Appendix: Off-Chain Reward Accounting

Please note that this mechanism is out-of-scope

The reward mechanism distributes a predefined reward pool across staked SushiSwap V3 positions based on in-range exposure.

The in-range exposure is derived from the Uniswap V3 pool accumulator secondsPerLiquidityInsideX128, which increases over time only when the pool price is inside the position's tick range [tickLower, tickUpper]. The accumulator is reported in Q128 fixed-point ("X128"), i.e. scaled by 2^128.

The staking contract emits:

- TokenStaked(..., liquidity, secondsPerLiquidityInsideInitialX128, timestamp)
- TokenUnstaked(..., liquidity, secondsPerLiquidityInsideX128, timestamp)

Off-chain, the difference between the two snapshots is used to compute a per-position weight, and rewards are distributed proportionally to this weight.

An example calculation is provided within the repository README.

Appendix: NFPM

Privileged Functions

- setGaugeVoter
- setFeeCollector

Core Invariants:

INV 1: Only tokenIDs which are related to the Sushiswap NFPM can be staked

INV 2: During the staking period, all rewards must be allocated to the feeCollector address

INV 3: Unstake is only callable if the tokenID has been staked

INV 4: Unstake is only callable by the original “from” address

INV 5: onERC721Received cannot be triggered in context of NFPM.mint

INV 6: Only the contract owner can call setFeeCollector and setGaugeVoter

INV 7: A position to stake must have non-zero liquidity

INV 8: Only valid pools (non-malicious) can be staked

Issue_01	Governance Privilege: Custody of tokenIDs
Severity	**Governance**
Description	Currently, governance of this contract has several privileges for invoking certain functions that can drastically alter the contracts behavior. For example, the proxy implementation can be upgraded which would then allow governance for withdrawing tokenIDs or the gaugeVoter can be set to a non-supportive address which would then revert upon the external call to fetch the epochID and hence reverts in fee claim / unstake.
Recommendations	Consider incorporating a Gnosis Multisignature contract as owner and ensuring that the Gnosis participants are trusted entities.
Comments / Resolution	Acknowledged.

Issue_02	A tokenID can remain permanently locked due to enforcement of safeTransferFrom
Severity	**High**
Description	The safeTransferFrom execution is an optional transfer method which on top of the transfer, invokes onERC721Received on the recipient address: *function checkOnERC721Received(* *address operator,* *address from,* *address to,* *uint256 tokenId,* *bytes memory data* *) internal {* *if (to.code.length > 0) {* *try IERC721Receiver(to).onERC721Received(operator, from, tokenId, data) returns (bytes4 retval) {* *if (retval != IERC721Receiver.onERC721Received.selector) {* *// Token rejected* *revert IERC721Errors.ERC721InvalidReceiver(to);* *}* *} catch (bytes memory reason) {* *if (reason.length == 0) {* *// non-IERC721Receiver implementer* *revert IERC721Errors.ERC721InvalidReceiver(to);* *} else {* *assembly ("memory-safe") {* *revert(add(reason, 0x20), mload(reason))* *}* *}* *}* *}* *}* Ironically, the “safe” mechanism is often “unsafe” as it often does more harm than good. While this is a helpful feature to prevent

	accidental loss of tokenIDs in case of unsupported contracts, it can backfire on many occasions. Consider the following scenario: 1. Alice mints tokenId = 10 via her smart wallet implementation, the tokenID is minted via standard _mint (not _safeMint): a. *_mint(params.recipient, (tokenId = _nextId++));* 2. Alice stakes the tokenID in the SushiStaker contract at t = 100 3. Alice unstakes the tokenID = 10 at t = 1000. The unstake function attempts a safeTransferFrom but Alice's smart wallet does not expose the onERC721Received function, because it was simply never required (the original _mint was fully supported) Since Alice's smart wallet is the msg.sender and that is hardcoded for tokenStaker[tokenID], any unstake revert will always attempt to transfer the tokenID to this address, which makes it permanently unretrievable.
Recommendations	Consider using the standard transferFrom function.
Comments / Resolution	Resolved.

Issue_03	Owner enforcement during stake can be bypassed via donation
Severity	**Low**
Description	The stake function ensures that only the owner can actually stake a tokenID: *// Verify the NFT is from the correct contract and user owns it* *if (IERC721(address($.sushiNft)).ownerOf(tokenId) != msg.sender) revert SushiStakerNotTokenOwner();* This can be trivially bypassed as follows: 1. Alice approves Bob to spend her tokenIDs 2. Bob calls safeTransferFrom with the SushiStaker being the recipient 3. It will invoke onERC721Received and successfully stakes the transfer The requirement that Alice must be the caller of the stake mechanism is now bypassed (Alice will still become the owner).
Recommendations	There is no real harm from such a behavior, however, we find it important to highlight violation of expected behaviors.
Comments / Resolution	Acknowledged.

Issue_04	Unstake might revert if feeCollector is blacklisted for token0/token1
Severity	Low
Description	Within the unstake function, fees are explicitly claimed to the feeCollector address: *// Collect fees accumulated during staking and send to feeCollector* *_collectAndTransferFees(tokenId, $.feeCollector, $);* *...* *(uint256 amount0, uint256 amount1) = $.sushiNft* *.collect(* *INonfungiblePositionManager.CollectParams({* *tokenId: tokenId, recipient: recipient, amount0Max: type(uint128).max, amount1Max: type(uint128).max* *})* *);* *...* *(amount0, amount1) = pool.collect(* *recipient,* *position.tickLower,* *position.tickUpper,* *amount0Collect,* *amount1Collect* *);* *...* *if (amount0 > 0) {* *position.tokensOwed0 -= amount0;* *TransferHelper.safeTransfer(token0, recipient, amount0);* *}* *if (amount1 > 0) {*

	position.tokensOwed1 -= amount1; *TransferHelper.safeTransfer(token1, recipient, amount1);* *}* If the feeCollector address is blacklisted for one of both tokens, while the amount is non-zero, the collect process will revert and so will the whole unstake process. The tokenId remains effectively locked until the feeCollector is changed. The same is applicable if one of the both underlying tokens is paused. If the owner held the tokenId, he could simply call collect with the blacklisted/paused token to be zero and at least get the other token out.
Recommendations	Consider keeping this scenario in mind and changing the feeCollector address immediately if such a blacklisting happens.
Comments / Resolution	Acknowledged.

Issue_05	Unexpected reentrancy concerns via direct token donation
Severity	**Low**
Description	In a previous issue, we have elaborated the ability to directly donate a tokenId to the contract to bypass the stake function call, which fully follows the _stakeInternal mechanism. This creates a non-obvious reentrancy vector, as during the fee collection, the msg.sender receives token0/token1. If one of both tokens (or both) is an ERC777 token, that exposes a reentrancy loophole where the msg.sender can hijack the control-flow and execute arbitrary logic before the actual stake accounting logic is being triggered: *_collectAndTransferFees(tokenId, staker, $);* *// Record staking info* *$.tokenStaker[tokenId] = staker;* *$.stakeTimestamp[tokenId] = block.timestamp;* The caller could for example stake another tokenId before the first call has been finalized. While we could not identify any specific harm, most of the time, exploits happen due to arbitrary user inputs or users invoking functions which are not meant to be invoked by users, one can argue that a large user flexibility is a great seed for exploits. Therefore, at BailSec, we are of the opinion that codebases should never provide more user flexibility than necessary during the normal business logic.
Recommendations	Consider further inspecting if there is any harm that can be done via that aforementioned control-flow. It has to be noted that it will not be practical to add a reentrancy guard to this function, as this would essentially block the standard stake control-flow. It should also be considered if that may have any unexpected impact on the off-chain calculation.

Comments / Resolution	Acknowledged.

Issue_06	Summary of totalLiquidity and totalTokensOwed0/1 is pointless for different pools
Severity	**Low**
Description	The collectFeesMultipleStats function aggregates liquidity and owed tokens via looping over all input tokenIDs. If these tokenIDs do not correspond to the same pool, this aggregation is fully incorrect.
Recommendations	Consider adding NATSPEC which highlights this constraint.
Comments / Resolution	Resolved, the function has been removed.

Issue_07	Staleness of collectFeesMultipleStats
Severity	**Informational**
Description	The aforementioned function invokes NFPM.positions which has the following implementation: *Position memory position = _positions[tokenId];* *require(position.poolId != 0, 'Invalid token ID');* *PoolAddress.PoolKey memory poolKey = _poolIdToPoolKey[position.poolId];* *return (* *position.nonce,* *position.operator,* *poolKey.token0,* *poolKey.token1,* *poolKey.fee,* *position.tickLower,* *position.tickUpper,* *position.liquidity,* *position.feeGrowthInside0LastX128,* *position.feeGrowthInside1LastX128,* *position.tokensOwed0,* *position.tokensOwed1* *);* The problem here is that feeGrowthInside0/1LastX128 and position.tokensOwed0/1 is stale, as there has not been any update on the NFPM after the tokenID has been staked into the SushiStaker contract. Unless someone calls NFPM.increaseLiquidity, these values will be fully stale and tokensOwed0/1 will always be zero. This can be problematic if there is an off-chain system built around the correctness of the view return value which then decides whether to claim fees from tokenIDs (or not). As this would then be incorrect due to the stale reporting.
Recommendations	Consider keeping this in mind, it would essentially require to

	execute a burn(0) on each position which is simply not possible for static execution (view-only environment).
Comments / Resolution	Resolved, the function has been removed.

Issue_08	Usage of normal reentrancy guard for proxy
Severity	**Informational**
Description	The SushiStaker contract is meant to be used as an implementation contract for a proxy. This is indicated by inheriting the Initializable contract and the OwnableUpgradeable contract. However, instead of inheriting ReentrancyGuardUpgradeable, the non-upgradeable version is inherited: *contract SushiStaker is Initializable, OwnableUpgradeable, **ReentrancyGuard**, IERC721Receiver*
Recommendations	While the usual recommendation would be to inherit the upgradeable version, we are of the opinion that this issue can be safely acknowledged, since it is never expected that there will be additional used storage slots in the reentrancy guard. In fact, it appears that the standard reentrancy guard now already uses the EIP-7201 pattern, which makes it safe to use. Reference: https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/utils/ReentrancyGuard.sol#L35C1-L37C76 While at the same time, it is notable that the upgradeable repository of OpenZeppelin contracts, does not contain the ReentrancyGuardUpgradeable contract anymore. See as comparison an older version: https://github.com/OpenZeppelin/openzeppelin-contracts-upgradeable/blob/5c4c29275d02e06265ce1cfcad5a420b58a5ca02/contracts/utils/ReentrancyGuardUpgradeable.sol
Comments / Resolution	Acknowledged.

Issue_09	Potential footguns during reward calculation
Severity	**Informational**
Description	The reward calculation is happening off-chain based on the events which have been triggered and the corresponding on-chain state at this specific time. The lightweight calculation example is as follows: *// Query events to get position data* *const stakedEvent = await contract.queryFilter('TokenStaked', ...);* *const { liquidity, secondsPerLiquidityInsideInitialX128 } = stakedEvent.args;* *// On unstake event* *const unstakedEvent = await contract.queryFilter('TokenUnstaked', ...);* *const { secondsPerLiquidityInsideX128: finalSecondsPerLiquidity } = unstakedEvent.args;* *// Calculate reward* *const secondsInside = finalSecondsPerLiquidity - secondsPerLiquidityInsideInitialX128;* *const rewardShare = (liquidity * secondsInside) / totalLiquiditySeconds;* *const userReward = totalRewardPool * rewardShare;* While this logic is fully out of scope, we are the opinion that it might be still from additional value for the protocol team if we provide certain footgun possibilities which should be avoided: a) The liquidity value should be the initial liquidity value and not the liquidity value during unstake. An attacker can call NFPM.increaseLiquidity to increase the liquidity value which would then falsify the reward calculation to the gain for the

	attacker. b) The off-chain calculation should be consistent with x128 scaling and corresponding arithmetic operations c) The x128 value might implicitly overflow within the UV3 implementation, this should be considered in the off-chain calculation d) The liquidityPerSecondsInsideX128 return value includes the full liquidity, including these positions which are not staked / do not have a tokenID (this is possible via direct Pool.mint call). This should be incorporated in the thought process. e) Reward calculation must incorporate the pool specific address (some pools, such as pools with novel tokens or unconventional fees, may not receive any rewards)
Recommendations	Consider being very cautious with the reward calculation mechanism and execute proper testing before execution. A fail-safe mechanism should be implemented in case of any manipulation attempts and corresponding addresses should not receive any rewards.
Comments / Resolution	Acknowledged.

Issue_10	Anti-mint guard is void
Severity	**Informational**
Description	The contract enforces the it cannot become the owner as result of a mint operation: *// Direct transfer - validate sender is not zero (prevents minting to contract)* *if (from == address(0)) revert SushiStakerZeroAddress();* This enforcement is however void, as the NFPM mints tokenIDs via _mint and not via _safeMint: *_mint(params.recipient, (tokenId = _nextId++));* The normal _mint operation does not trigger onERC721Received which means that this check is effectively unreachable code.
Recommendations	Consider acknowledging this note, minting a tokenID or transferring it via transferFrom is considered as a user mistake.
Comments / Resolution	Acknowledged.